

NutriAgent: Multi-Agent AI Nutrition Advisor

System Architecture, Agent Specifications, Shared Contracts & Implementation Roadmap

Course: IT3041 — Information Retrieval & Web Analytics | Institution: SLIIT | Architecture: 4-Agent Microservice Pipeline

1. Executive Summary & Purpose

NutriAgent is a multi-agent artificial intelligence nutrition advisory system designed to convert messy, free-text dietary requirements, allergies, medical conditions, and wellness goals into personalized, explainable, and factually grounded meal plans. Unlike conventional black-box LLM systems that frequently hallucinate nutritional values and calorie counts, NutriAgent grounds all reasoning in verifiable nutrition datasets retrieved through dense semantic vector search.

The system is decomposed into **four independent microservices (agents)**, each assigned to a team member and communicating over synchronous HTTP with strict JSON schema contracts. There is no central orchestrator or message broker; communication follows a direct chained request-response flow.

2. System Architecture & Request-Response Pipeline

The architecture follows a synchronous chained pipeline designed for modularity, testability, and clear separation of concerns:

Stage	Agent & Port	Role & Responsibilities	Data Input / Output
1. Gateway	Security Agent Port 8001	System front-door. Handles authentication, prompt injection detection, input sanitization, rate limiting, and structured audit logging.	In: SecurityRequest Out: MealPlan (forwarded)
2. Extraction	Intake Agent Port 8002	Natural Language Processing engine. Extracts structured entities (goals, allergies, conditions, diet types) into a validated UserProfile.	In: IntakeRequest Out: UserProfile, coordinates downstream
3. Retrieval	Nutrition IR Agent Port 8003	Ground truth knowledge layer. Performs dense vector search (ChromaDB + Sentence-Transformers) on USDA food data and applies strict allergen/diet safety filters.	In: IRRequest Out: IRResponse (FoodItem list)
4. Reasoning	Meal Planning Agent Port 8004	Explainable reasoning layer. Uses LLM (Claude/OpenAI) to reason over the retrieved food candidates and construct a transparent, explainable meal plan.	In: PlanningRequest Out: MealPlan

3. Shared JSON Contract (shared/schemas.py)

To guarantee zero schema drift between agents, all models are centralized in shared/schemas.py. No agent defines local duplicate schemas.

Schema Model	Key Fields & Data Types	Usage Context
UserProfile	user_id: str, goals: List[str], allergies: List[str], conditions: List[str], diet_type: Optional[str], calorie_target: Optional[int], preferences: List[str]	Core user representation created by Intake Agent and consumed by IR & Planning Agents.
FoodItem	name: str, calories: float, protein_g: Optional[float], carbs_g: Optional[float], fat_g: Optional[float], source: str	Individual factual food/nutrition record returned by the IR Agent.
IRRequest / IRResponse	IRRequest(profile: UserProfile, query: str) IRResponse(items: List[FoodItem])	Data contract for semantic retrieval and constraint filtering.
MealPlan	user_id: str, meals: List[MealRecommendation(name, calories, reason, source)], disclaimer: str	Final explainable response returned back up through the chain to the client.

4. Individual Agent Specifications & Team Allocation

Member 1: Security & Validation Agent (Port 8001)

Primary Role: API gateway, security perimeter, authentication, and sanitization.

- **Implemented / Current:** FastAPI service, /health check, regex-based prompt-injection blocking (e.g. 'ignore instructions', 'system prompt'), HTTP forwarding to Intake.
- **Deliverables & Next Steps:** Real JWT verification (pyjwt/python-jose), Fernet field-level encryption (cryptography.fernet) for health data, rate limiting with slowapi, structured trace-ID request logging.

Member 2: Intake & Profile Agent (Port 8002)

Primary Role: Entity extraction and downstream pipeline coordination.

- **Implemented / Current:** FastAPI service, /health endpoint, sequential forwarding to IR Agent and Planning Agent.
- **Deliverables & Next Steps:** Replace keyword stub with real Named Entity Recognition (spaCy custom NER or LLM JSON extractor), profile persistence (SQLite), intent classification (new profile vs update).

Member 3: Nutrition Information Retrieval (IR) Agent (Port 8003)

Primary Role: Ground truth retrieval, vector indexing, and deterministic safety filtering.

- **Fully Implemented & Verified:**
 1. **Data Pipeline:** Cleaned USDA Foundation Foods (1,178 items) + curated balanced dishes with exact calories and macronutrients (protein, carbs, fat).

2. **Vector Engine:** Embedded ChromaDB vector database using sentence-transformers (all-MiniLM-L6-v2, 384 dimensions).
3. **Two-Stage Retrieval:** Contextual query enrichment -> Cosine similarity vector search -> Strict deterministic allergy/diet post-filtering (peanuts, gluten, dairy, vegan, keto).
4. **Verification:** Standalone test suite (test_ir_agent.py) and live FastAPI endpoints validated.

Member 4: Meal Planning Agent (Port 8004)

Primary Role: Explainable reasoning and meal recommendation synthesis.

• **Implemented / Current:** FastAPI service returning validated MealPlan structure.

- **Deliverables & Next Steps:** LLM integration (Anthropic Claude or OpenAI SDK), prompt grounding strictly constrained to request.retrieved_items, generating explicit explainable 'reason' strings tied back to user profile fields, safety guardrail against medical diagnosis.

5. Execution Guide & Best Practices

Running the Services: Open 4 terminals and launch from repository root:

```
# Terminal 1: uvicorn security_agent.main:app --reload --port 8001
# Terminal 2: uvicorn intake_agent.main:app --reload --port 8002
# Terminal 3: uvicorn ir_agent.main:app --reload --port 8003
# Terminal 4: uvicorn planning_agent.main:app --reload --port 8004
```

End-to-End Smoke Test Command:

```
curl -X POST http://localhost:8001/process \
  -H "Content-Type: application/json" \
  -d '{"user_id": "u123", "raw_text": "I want to gain muscle, I am allergic to peanuts"}'
```

6. Project Evaluation Checklist

- **Architectural Integrity:** Microservices maintain independence with zero circular dependencies and consistent schema adherence.
- **Ground Truth Information Retrieval:** IR Agent eliminates hallucinated nutrition by indexing real USDA FoodData Central records.
- **Responsible AI & Explainability:** Recommendations cite actual source datasets and provide clear user-centric justifications.
- **Course Deliverables:** Modular code repository, API documentation at /docs per agent, and full test suite.